Tutorial 8

Name: Mahalakshmi A
RegNo: RA2511033010116
Section: AM2

1. Hospital Emergency Monitoring System: A hospital monitoring application uses three threads: EmergencyAlert for critical patient alerts, VitalMonitor for continuously checking vital signs, and ReportGenerator for preparing routine reports. Develop a Java multithreaded program that assigns the highest priority to EmergencyAlert, a medium priority to VitalMonitor, and the lowest priority to ReportGenerator. Display the name and priority of each thread during execution using getName() and getPriority().

```java
class EmergencyAlert extends Thread {  2 usages
    public void run() {
        System.out.println("Thread Name: " + getName());
        System.out.println("Priority: " + getPriority());
        System.out.println("Monitoring critical patient alerts...\n");
    }
}

class VitalMonitor extends Thread {  2 usages
    public void run() {
        System.out.println("Thread Name: " + getName());
        System.out.println("Priority: " + getPriority());
        System.out.println("Checking patient vital signs...\n");
    }
}

class ReportGenerator extends Thread {  2 usages
    public void run() {
        System.out.println("Thread Name: " + getName());
        System.out.println("Priority: " + getPriority());
        System.out.println("Generating routine reports...\n");
    }
}

public class HospitalMonitoring {
    public static void main(String[] args) {

        EmergencyAlert emergency = new EmergencyAlert();
        VitalMonitor vital = new VitalMonitor();
        ReportGenerator report = new ReportGenerator();
```

```java
public class HospitalMonitoring {
    public static void main(String[] args) {

        EmergencyAlert emergency = new EmergencyAlert();
        VitalMonitor vital = new VitalMonitor();
        ReportGenerator report = new ReportGenerator();

        emergency.setName("EmergencyAlert");
        vital.setName("VitalMonitor");
        report.setName("ReportGenerator");


        emergency.setPriority(Thread.MAX_PRIORITY);
        vital.setPriority(Thread.NORM_PRIORITY);
        report.setPriority(Thread.MIN_PRIORITY);

        emergency.start();
        vital.start();
        report.start();
    }
}
```

Output:

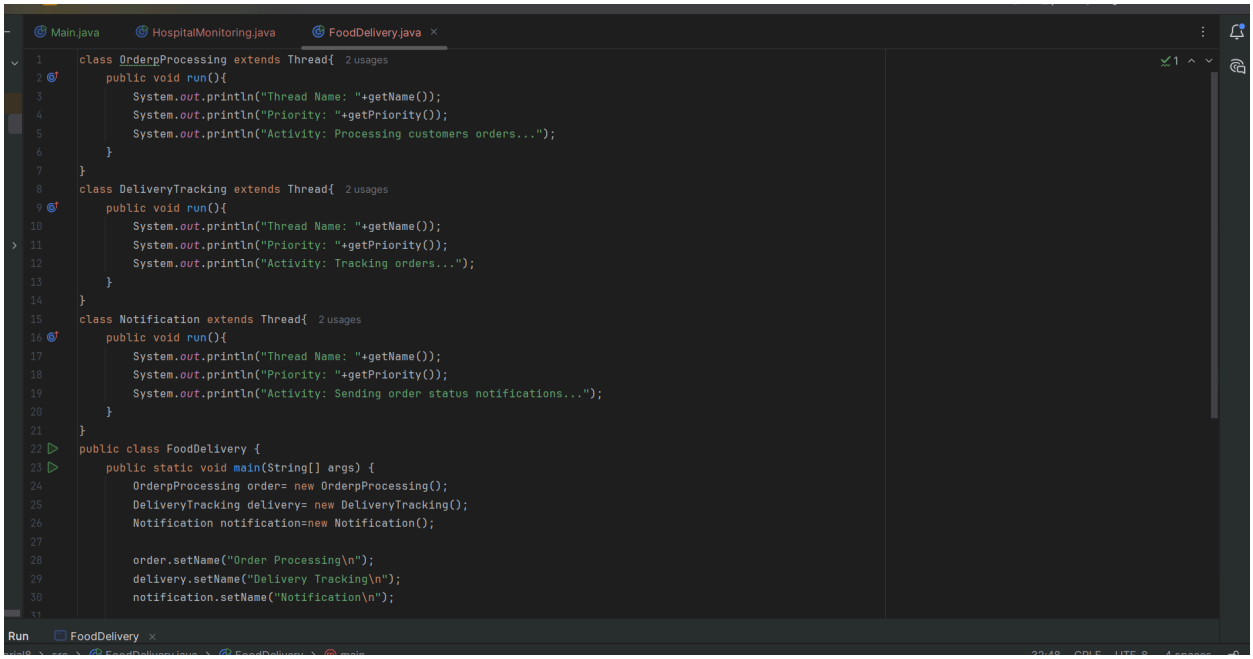
```
C:\Users\Sahaana\.jdks\openjdk-14.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 202
Thread Name: EmergencyAlert
Thread Name: VitalMonitor
Priority: 5
Checking patient vital signs...

Priority: 10
Monitoring critical patient alerts...

Thread Name: ReportGenerator
Priority: 1
Generating routine reports...


Process finished with exit code 0
```

2. An online food delivery application performs multiple tasks concurrently, such as processing customer orders, tracking delivery locations, and sending order-status notifications. Develop a Java program using multithreading by creating three separate threads named OrderProcessing, DeliveryTracking, and Notification. Assign different priorities to the three threads based on the importance of their tasks using setPriority(). Start all the threads from the main method and display the thread name, assigned priority, and activity being performed to demonstrate multiple thread creation, thread priority, and concurrent execution.



```
Main.java  HospitalMonitoring.java  FoodDelivery.java

class OrderpProcessing extends Thread{  2 usages
    public void run(){
        System.out.println("Thread Name: "+getName());
        System.out.println("Priority: "+getPriority());
        System.out.println("Activity: Processing customers orders...");
    }
}
class DeliveryTracking extends Thread{  2 usages
    public void run(){
        System.out.println("Thread Name: "+getName());
        System.out.println("Priority: "+getPriority());
        System.out.println("Activity: Tracking orders...");
    }
}
class Notification extends Thread{  2 usages
    public void run(){
        System.out.println("Thread Name: "+getName());
        System.out.println("Priority: "+getPriority());
        System.out.println("Activity: Sending order status notifications...");
    }
}
public class FoodDelivery {
    public static void main(String[] args) {
        OrderpProcessing order= new OrderpProcessing();
        DeliveryTracking delivery= new DeliveryTracking();
        Notification notification=new Notification();

        order.setName("Order Processing\n");
        delivery.setName("Delivery Tracking\n");
        notification.setName("Notification\n");

Run  FoodDelivery
```

```
        order.setPriority(Thread.MAX_PRIORITY);
        delivery.setPriority(Thread.NORM_PRIORITY);
        notification.setPriority(Thread.MIN_PRIORITY);

        order.start();
        delivery.start();
        notification.start();

    }
```

Output:

```
C:\Users\Sahaana\.jdks\openjdk-14.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition
Thread Name: Order Processing

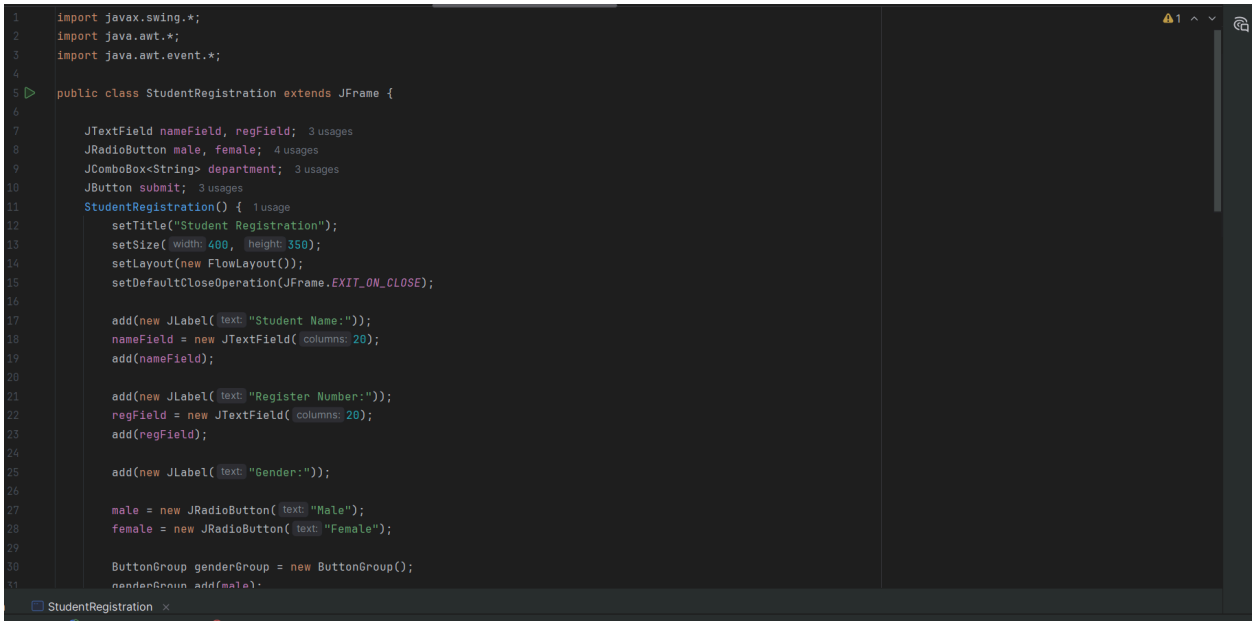
Priority: 10
Activity: Processing customers orders...
Thread Name: Delivery Tracking

Priority: 5
Activity: Tracking orders...
Thread Name: Notification

Priority: 1
Activity: Sending order status notifications...

Process finished with exit code 0
```

3. Student Registration System: Develop a Java Swing application for student registration using JFrame. Provide JTextField components to enter the student name and register number, JRadioButton components to select gender, and JComboBox to select the department. Include a JButton to submit the details and display the entered information using a JOptionPane.



```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class StudentRegistration extends JFrame {

    JTextField nameField, regField;  3 usages
    JRadioButton male, female;  4 usages
    JComboBox<String> department;  3 usages
    JButton submit;  3 usages
    StudentRegistration() {  1 usage
        setTitle("Student Registration");
        setSize( width: 400,  height: 350);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        add(new JLabel( text: "Student Name:"));
        nameField = new JTextField( columns: 20);
        add(nameField);

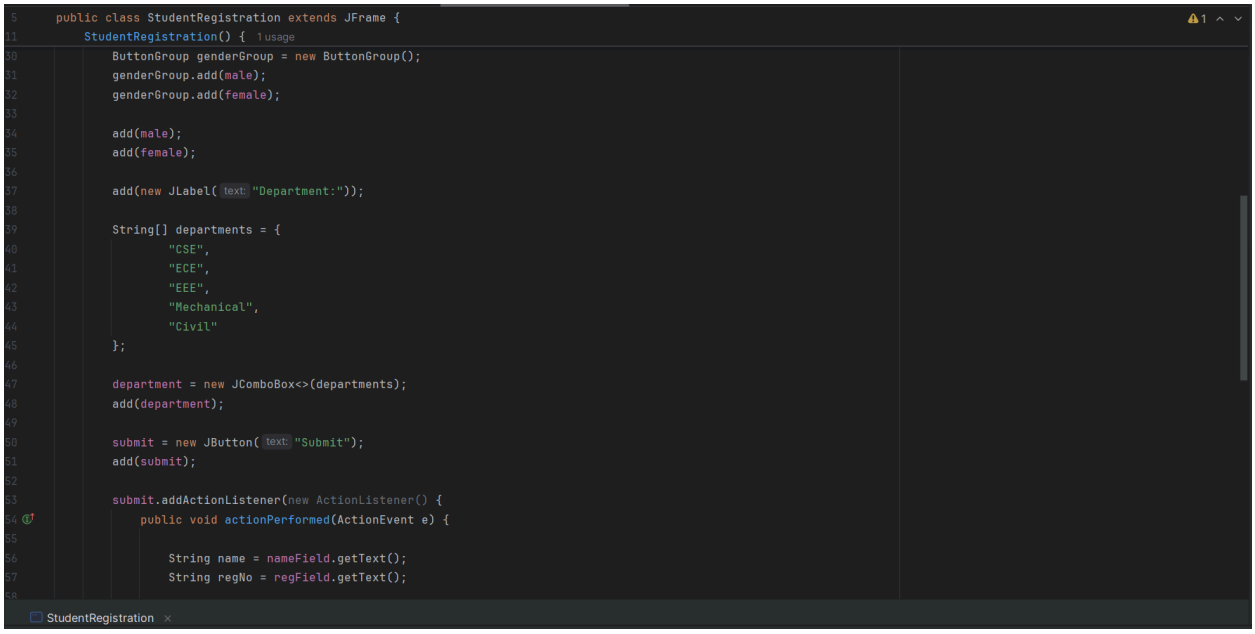
        add(new JLabel( text: "Register Number:"));
        regField = new JTextField( columns: 20);
        add(regField);

        add(new JLabel( text: "Gender:"));

        male = new JRadioButton( text: "Male");
        female = new JRadioButton( text: "Female");

        ButtonGroup genderGroup = new ButtonGroup();
        genderGroup.add(male);
```

StudentRegistration



```java
public class StudentRegistration extends JFrame {
    StudentRegistration() {  1 usage
        ButtonGroup genderGroup = new ButtonGroup();
        genderGroup.add(male);
        genderGroup.add(female);

        add(male);
        add(female);

        add(new JLabel( text: "Department:"));

        String[] departments = {
                "CSE",
                "ECE",
                "EEE",
                "Mechanical",
                "Civil"
        };

        department = new JComboBox<>(departments);
        add(department);

        submit = new JButton( text: "Submit");
        add(submit);

        submit.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {

                String name = nameField.getText();
                String regNo = regField.getText();
```

StudentRegistration

```java
public class StudentRegi  C:\Users\Sahaana\IdeaProjects\Tutorial8\src\HospitalMonitoring.java
    StudentRegistration() {  1 usage
        submit.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String name = nameField.getText();
                String regNo = regField.getText();

                String gender;

                if (male.isSelected())
                    gender = "Male";
                else if (female.isSelected())
                    gender = "Female";
                else
                    gender = "Not Selected";

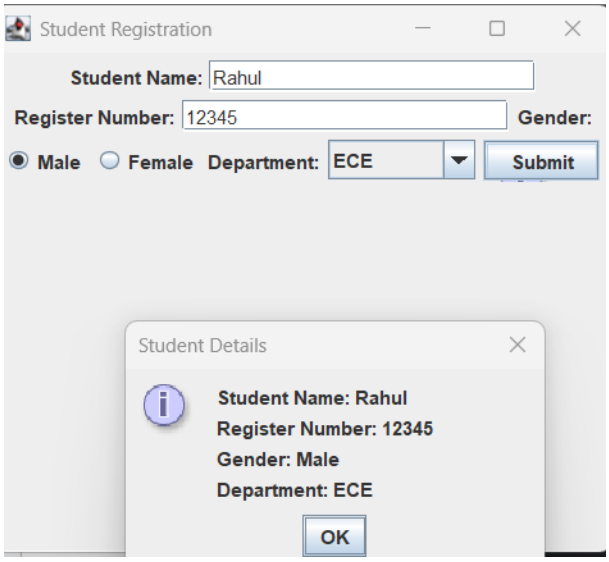
                String dept = (String) department.getSelectedItem();

                JOptionPane.showMessageDialog(
                        parentComponent: StudentRegistration.this,
                        message: "Student Name: " + name +
                                "\nRegister Number: " + regNo +
                                "\nGender: " + gender +
                                "\nDepartment: " + dept,
                        title: "Student Details",
                        JOptionPane.INFORMATION_MESSAGE
                );
            }
        });
```

```java
        setVisible(true);
    }

    public static void main(String[] args) {
        new StudentRegistration();
    }
}
```

Output:



4. User Login and Preferences: Develop a Java Swing application for a user login system. Use JTextField to enter the username and JPasswordField to enter the password. Provide JCheckBox components for selecting user preferences such as Remember Me and Receive Notifications. On clicking the Login button, display an appropriate login message using JOptionPane.

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class UserLogin extends JFrame {

    JTextField usernameField;  3 usages
    JPasswordField passwordField;  3 usages
    JCheckBox rememberMe, notifications;  3 usages
    JButton loginButton;  3 usages

    UserLogin() {  1 usage

        setTitle("User Login");
        setSize( width: 400,  height: 300);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        add(new JLabel( text: "Username:"));
        usernameField = new JTextField( columns: 20);
        add(usernameField);

        add(new JLabel( text: "Password:"));
        passwordField = new JPasswordField( columns: 20);
        add(passwordField);

        rememberMe = new JCheckBox( text: "Remember Me");
        notifications = new JCheckBox( text: "Receive Notifications");

        add(rememberMe);
```

```java
public class UserLogin extends JFrame {
    UserLogin() {  1 usage
        add(rememberMe);
        add(notifications);

        loginButton = new JButton( text: "Login");
        add(loginButton);

        loginButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {

                String username = usernameField.getText();
                String password = new String(passwordField.getPassword());

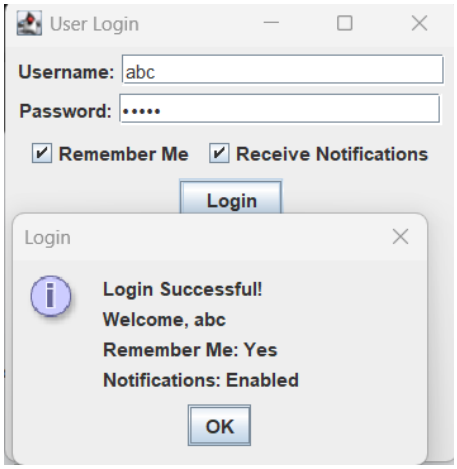
                if (username.isEmpty() || password.isEmpty()) {
                    JOptionPane.showMessageDialog(
                            parentComponent: UserLogin.this,
                            message: "Please enter username and password.",
                            title: "Login",
                            JOptionPane.WARNING_MESSAGE
                    );
                } else {
                    String message = "Login Successful!\n"
                            + "Welcome, " + username + "\n"
                            + "Remember Me: "
                            + (rememberMe.isSelected() ? "Yes" : "No") + "\n"
                            + "Notifications: "
                            + (notifications.isSelected() ? "Enabled" : "Disabled");

                    JOptionPane.showMessageDialog(
```

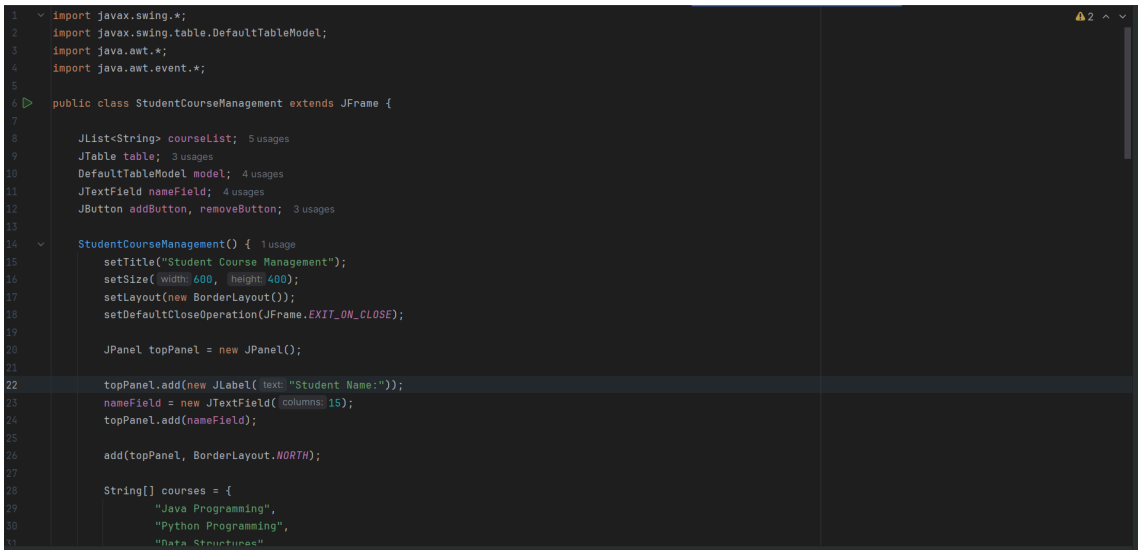
```
                    JOptionPane.showMessageDialog(
                            parentComponent: UserLogin.this,
                            message,
                            title: "Login",
                            JOptionPane.INFORMATION_MESSAGE
                    );
                }
            }
        });

        setVisible(true);
    }

    public static void main(String[] args) {
        new UserLogin();
    }
}
```



5. Student Course Management System: Design a Java Swing application to maintain student course information. Use a JList to display the available courses and a JTable to display the student name, selected course, and enrollment status. Provide suitable JButton components to add and remove course registrations and use a JScrollPane wherever required to handle larger amounts of information.



```java
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;

public class StudentCourseManagement extends JFrame {

    JList<String> courseList;  5 usages
    JTable table;  3 usages
    DefaultTableModel model;  4 usages
    JTextField nameField;  4 usages
    JButton addButton, removeButton;  3 usages

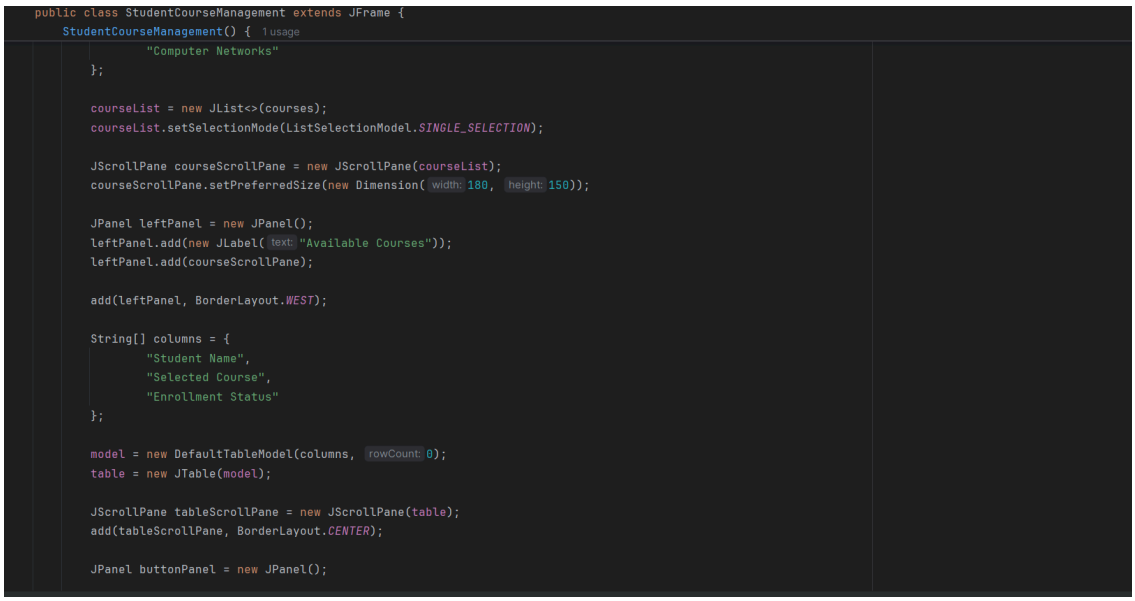
    StudentCourseManagement() {  1 usage
        setTitle("Student Course Management");
        setSize( width: 600,  height: 400);
        setLayout(new BorderLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel topPanel = new JPanel();

        topPanel.add(new JLabel( text: "Student Name:"));
        nameField = new JTextField( columns: 15);
        topPanel.add(nameField);

        add(topPanel, BorderLayout.NORTH);

        String[] courses = {
                "Java Programming",
                "Python Programming",
                "Data Structures"
```



```java
public class StudentCourseManagement extends JFrame {
    StudentCourseManagement() {  1 usage
                "Computer Networks"
        };

        courseList = new JList<>(courses);
        courseList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

        JScrollPane courseScrollPane = new JScrollPane(courseList);
        courseScrollPane.setPreferredSize(new Dimension( width: 180,  height: 150));

        JPanel leftPanel = new JPanel();
        leftPanel.add(new JLabel( text: "Available Courses"));
        leftPanel.add(courseScrollPane);

        add(leftPanel, BorderLayout.WEST);

        String[] columns = {
                "Student Name",
                "Selected Course",
                "Enrollment Status"
        };

        model = new DefaultTableModel(columns,  rowCount: 0);
        table = new JTable(model);

        JScrollPane tableScrollPane = new JScrollPane(table);
        add(tableScrollPane, BorderLayout.CENTER);

        JPanel buttonPanel = new JPanel();
```

```
public class StudentCourseManagement extends JFrame {
    StudentCourseManagement() {  1 usage
        JPanel buttonPanel = new JPanel();

        addButton = new JButton( text: "Add Registration");
        removeButton = new JButton( text: "Remove Registration");

        buttonPanel.add(addButton);
        buttonPanel.add(removeButton);

        add(buttonPanel, BorderLayout.SOUTH);

        addButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String name = nameField.getText();
                String course = courseList.getSelectedValue();

                if (name.isEmpty() || course == null) {
                    JOptionPane.showMessageDialog(
                            parentComponent: StudentCourseManagement.this,
                            message: "Enter student name and select a course."
                    );
                } else {
                    model.addRow(new Object[]{
                            name,
                            course,
                            "Enrolled"
                    });

                    nameField.setText("");
                    courseList.clearSelection();
```

```
public class StudentCourseManagement extends JFrame {
    StudentCourseManagement() {  1 usage
        addButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                    nameField.setText("");
                    courseList.clearSelection();
                }
            }
        });

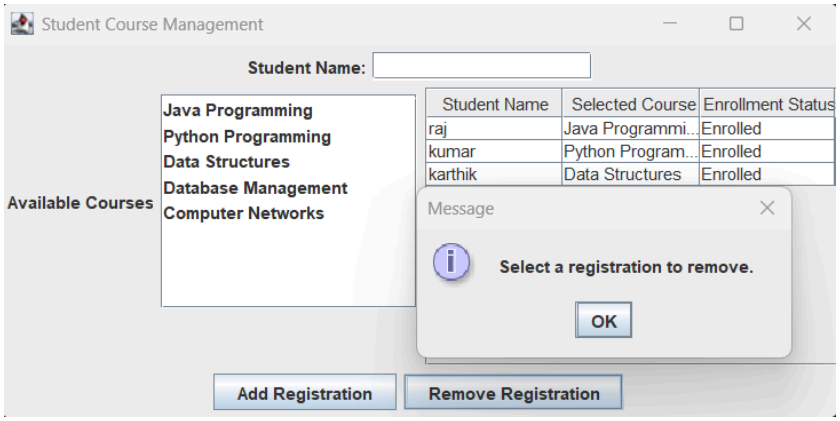
        removeButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                int row = table.getSelectedRow();

                if (row != -1) {
                    model.removeRow(row);
                } else {
                    JOptionPane.showMessageDialog(
                            parentComponent: StudentCourseManagement.this,
                            message: "Select a registration to remove."
                    );
                }
            }
        });

        setVisible(true);
    }

    public static void main(String[] args) {
        new StudentCourseManagement();
    }
```

Output:



6. Text Editor Application: Develop a simple Java Swing-based text editor using JTextArea for entering and editing text. Provide a JMenuBar containing a File menu with JMenuItem options such as New, Clear, and Exit, and an Edit menu with appropriate editing options. Place the text area inside a JScrollPane to allow scrolling when the text exceeds the visible area.

```
import javax.swing.*;
import java.awt.*;
public class TextEditor extends JFrame {
    JTextArea textArea;  7 usages
    TextEditor() {  1 usage
        setTitle("Text Editor");
        setSize( width: 500, height: 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        textArea = new JTextArea();
        JScrollPane scrollPane = new JScrollPane(textArea);
        add(scrollPane, BorderLayout.CENTER);
        JMenuBar menuBar = new JMenuBar();
        JMenu fileMenu = new JMenu( s: "File");
        JMenu editMenu = new JMenu( s: "Edit");
        JMenuItem newItem = new JMenuItem( text: "New");
        JMenuItem clearItem = new JMenuItem( text: "Clear");
        JMenuItem exitItem = new JMenuItem( text: "Exit");
        JMenuItem cutItem = new JMenuItem( text: "Cut");
        JMenuItem copyItem = new JMenuItem( text: "Copy");
        JMenuItem pasteItem = new JMenuItem( text: "Paste");
        fileMenu.add(newItem);
        fileMenu.add(clearItem);
        fileMenu.add(exitItem);
        editMenu.add(cutItem);
        editMenu.add(copyItem);
        editMenu.add(pasteItem);
        menuBar.add(fileMenu);
        menuBar.add(editMenu);
        setJMenuBar(menuBar);
        newItem.addActionListener( ActionEvent e -> textArea.setText(""));
        clearItem.addActionListener( ActionEvent e -> textArea.setText(""));
        exitItem.addActionListener( ActionEvent e -> System.exit( status: 0));
        cutItem.addActionListener( ActionEvent e -> textArea.cut());
        copyItem.addActionListener( ActionEvent e -> textArea.copy());
        pasteItem.addActionListener( ActionEvent e -> textArea.paste());
        setVisible(true);
    }
    public static void main(String[] args) {
        new TextEditor();
    }
}
```

Output:

